TECHNICAL ARCHITECTURE DOCUMENT

IGT Slot Audio Manager

Cross-Platform Electron Desktop Application for Slot Game Audio Workflow

VERSION	DATE
2.0	April 2026
AUTHOR	**AUDIENCE**
Bojan Petkovic	Tech Leads & Architecture Review

TABLE OF CONTENTS

01	Executive Summary	Overview & impact
02	Technology Stack	Electron, React, Vite, FFmpeg
03	Architecture Overview	Process model, page architecture
04	Application Pages — Detailed Walkthrough	All 7 tabs explained
05	Data Model	JSON schemas, project structure
06	Tiered Audio Loading	SubLoader system, pool architecture
07	Build Pipeline	WAV → M4A → deploy flow
08	IPC Channel Reference	30+ channels documented
09	Security Model	Process isolation, input validation
10	Cross-Platform Compatibility	macOS + Windows
11	UI Design System	Dark theme, components, animations
12	Build & Distribution	DMG, NSIS, template system
13	Key Design Decisions	Rationale for architectural choices

IGT Slot Audio Manager — Technical Architecture Document

Version: 2.0 **Date:** April 2026 **Author:** Bojan Petkovic **Audience:** Tech Leads, Architecture Review Board, Engineering Management

---

1. Executive Summary

IGT Slot Audio Manager is a cross-platform Electron desktop application that manages the complete audio workflow for IGT slot games built on the playa-core framework. It replaces a fragmented, manual, error-prone process — where audio engineers juggle CLI tools, JSON files, and multiple repositories — with a single integrated tool covering the entire lifecycle: import, configure, build, validate, deploy, and launch.

Key metrics:

- ~9,200 lines of source code across 23 files
- 7-page single-window UI with always-mounted architecture
- 30+ IPC channels between main and renderer processes
- Supports tiered audio sprite building with SubLoader integration
- Full git integration for both audio and game repositories
- Cross-platform: macOS (DMG) + Windows (NSIS installer)

Before this tool existed:

1. Audio engineer manually runs 3–5 CLI scripts in sequence
2. Edits JSON files by hand (typo → broken audio in production)
3. Copies files between repos manually
4. Launches game via separate terminal commands
5. Git operations in a separate terminal

With this tool:

1. One click: Build → Deploy → Launch
2. Visual editors for all JSON configurations
3. Automatic validation catches errors before deploy
4. Integrated git workflow with auto-generated commit messages and PRs

---

2. Technology Stack

LAYER	TECHNOLOGY	VERSION	PURPOSE
Runtime	Electron	28	Desktop shell, main process, IPC
Renderer	React	19	UI components, state management
Styling	Tailwind CSS	v4	Utility-first CSS with custom dark theme
Bundler	Vite	8	Renderer build, HMR in dev
Audio	FFmpeg (static)	bundled	WAV → M4A encoding
Audio	audiosprite	npm	Sprite file concatenation
Audio	node-exiftool	npm	Duration extraction from audio metadata
Audio	sox	npm	Audio processing and analysis
Packaging	electron-builder	latest	DMG (macOS), NSIS (Windows) installers

Language: Pure JavaScript — CommonJS in main/preload, ESM (JSX) in renderer. No TypeScript by design for rapid iteration and minimal build overhead.

3. Architecture Overview

```
┌──────────────────────────────────────────────────────────┐
│                    Electron Main Process                  │
│                       main.js (1,676 lines)               │
│                                                           │
│  ┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────┐ │
│  │ File Ops │  │ Git Ops  │  │ Process  │  │ IPC Handlers│ │
│  │ JSON R/W │  │ Status   │  │ Spawning │  │ 30+ channels│ │
│  │ WAV Mgmt │  │ Commit   │  │ npm/yarn │  │ Validation  │ │
│  │ Template │  │ Push/PR  │  │ playa    │  │ Security    │ │
│  └──────────┘  └──────────┘  └──────────┘  └──────────┘ │
│                                                           │
│  contextIsolation: true       nodeIntegration: false      │
├──────────────────────────────────────────────────────────┤
│                    preload.js (bridge)                    │
│             contextBridge.exposeInMainWorld('api')        │
├──────────────────────────────────────────────────────────┤
│                   Electron Renderer Process               │
│                                                           │
│  ┌──────────────────────────────────────────────────┐    │
│  │                     App.jsx                      │    │
│  │  Sidebar Nav | Always-mounted pages | Toast system    │    │
│  ├──────────────────────────────────────────────────┤    │
│  │  ProjectPage | SetupPage | SoundsPage | SpriteConfig  │    │
│  │  CommandsPage | BuildPage | GitPage              │    │
│  └──────────────────────────────────────────────────┘    │
│                                                           │
│  React 19 + Tailwind v4 + Vite 8                          │
└──────────────────────────────────────────────────────────┘
```

3.1 Process Model

The application follows Electron's two-process model with strict security boundaries:

- **Main process** (`main.js`): All file system operations, git commands, subprocess spawning, and IPC handling. No renderer code touches Node.js APIs.
- **Renderer process** (`src/`): React UI with no access to Node.js. All operations are requested via `window.api.*` which maps to IPC invoke calls.
- **Preload bridge** (`preload.js`): Thin layer exposing 40+ API methods via `contextBridge`. Each method maps 1:1 to an IPC channel.

3.2 Always-Mounted Page Architecture

All 7 pages are **always mounted** simultaneously. Navigation toggles `display: none` — pages are never unmounted. This preserves state across page switches (build logs, form inputs, scroll positions, audio playback).

```
App.jsx renders ALL 7 pages simultaneously
  ├── active page:   display: block  (visible)
  └── inactive pages: display: none   (hidden, state preserved)
```

Why this matters:

- Build log in BuildPage accumulates even when user is on another tab
- Audio playback in SoundsPage continues across navigation
- Form state (unsaved edits) persists without serialization
- No re-mount cost on tab switch — instant navigation

---

4. Application Pages — Detailed Walkthrough

4.1 Project Page — Dashboard & Overview

> *The landing page. Shows project health at a glance and provides the entry point for linking a game repository.*

Layout: Hero welcome screen (no project) → Dashboard grid (project loaded)

Sections:

SECTION	CONTENT
Stat Cards (4)	WAV file count, Commands count, Sprites count, Unassigned sounds count
Game Repo Panel	Linked game path, repo existence status, node_modules status
Last Build Panel	Sprite count, total size (MB), file listing from dist/

Key Actions:

- `Open Project` — File dialog to select audio repo folder
- `Reload` — Re-reads all config files from disk
- `Link Game Repository` — Picks game repo, auto-syncs template, auto-installs dependencies

Example flow:

```
User clicks "Open Project"
  → Dialog selects: C:\IGT\audio-treasure-of-troy
  → App loads: package.json, settings.json, sounds.json, sprite-config.json
  → Dashboard shows: 47 WAVs, 32 commands, 3 sprites, 5 unassigned
  → Game Repo: "../treasure-of-troy" ✓ exists, ✓ node_modules
```

Visual indicators:

- Green dot = healthy, Orange dot = needs attention, Red dot = missing
- Color-coded stat cards with icons
- Truncated paths with monospace font

4.2 Setup Page — Project Initialization & Dependencies

> *Configures the audio project from the bundled template and manages game repo dependencies.*

Layout: Two-column grid

LEFT COLUMN	RIGHT COLUMN
Sync Template (with mode selection)	Game Branch Selector
Sync log with line-by-line status	Pull sounds.json from Game
	Game Dependencies (yarn install)

Template Sync Modes:

1. **Sync All** — Overwrites scripts, configs (sprite-config.json, sounds.json), and merges package.json
2. **Skip Configs** — Overwrites scripts only, preserves existing configs (recommended after first init)

Example sync log:

```
✓ Copied 12 scripts to scripts/
✓ Merged package.json (3 scripts, 4 dependencies)
✓ Updated settings.json (preserved gameProjectPath)
⚠ Skipped sprite-config.json (Skip Configs mode)
⚠ Skipped sounds.json (Skip Configs mode)
✓ Created sourceSoundFiles/ directory
✓ Pulled sounds.json from game deploy path
```

Game Branch Selector:

- Dropdown with remote branches
- Switch & Pull button — checks out branch, pulls latest
- Auto-detects Node version from nvm
- Badge shows current branch and Node version

Key Actions:

- `Sync Template` → `npm install` (auto-chained)
- `Switch & Pull` — Changes game repo branch
- `Pull from Game` — Copies sounds.json from game's deploy path to audio repo root
- `Install` — Runs `yarn install` in game repo

4.3 Sounds Page — Source WAV File Management

> *Manages the raw WAV source files that become audio sprites. Full playback, waveform visualization, and JSON integration.*

Layout: Header with actions → Search filter → Sound list grid → Modals

Sound List Grid (5 columns):

COLUMN	CONTENT
Play/Pause	Web Audio playback with waveform visualization
Filename	Monospace name, truncated
JSON Status	Green "in JSON" badge or orange "+ Add" button
Size	File size in KB (right-aligned)
Delete	Trash icon (visible on hover)

Waveform Player:

- Custom `decodeWav()` function — pure JS, supports 8/16/24/32-bit PCM and 32-bit IEEE float
- Never uses `decodeAudioData()` (crashes Electron for 24-bit and 32-bit int PCM)
- 200-peak waveform visualization with animated playhead
- Click-to-seek on waveform bar
- Pause/resume with accurate position tracking

JSON Cleanup (Orphan Analysis):

```
Analyze → Finds sprites in sounds.json with no matching WAV file

Example result:
  3 orphaned sprites: s_DeletedSound1, s_OldEffect, s_Unused
  2 affected sprite lists: sl_WinSounds (1 item), sl_Bonus (1 item)
  1 affected command: onOldEvent (2 steps)

  → "Delete 3 orphan(s)" removes dead references from sounds.json
```

Key Actions:

- `Import WAVs` — Multi-select file dialog, copies WAVs to sourceSoundFiles/
- `Add All to JSON` — Bulk adds all unassigned sounds with auto-tag detection
- `Analyze` / `Delete orphans` — Finds and cleans dead references
- `Trash` — Opens trash modal showing soft-deleted files (restorable)
- Play/Pause/Stop — Per-sound audio playback with visual feedback

Soft Delete System:

- Delete moves WAV to `sourceSoundFiles/.deleted/`
- Trash modal lists deleted files with Restore buttons
- Permanent deletion only happens when `.deleted/` is manually cleared

---

4.4 Sprite Config Page — Audio Tier Configuration

> *Assigns sounds to loading pools (tiers) that control when and how audio is loaded at runtime.*

Layout: Header with unassigned count → Pool cards (one per tier) → Standalone card

The 4-Pool Architecture:

```
┌──────────────┐  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐
│  LOADING     │  │    MAIN      │  │    BONUS     │  │  STANDALONE  │
│  Immediate   │  │ Deferred A   │  │ Deferred B   │  │  Separate    │
│              │  │              │  │ Unloadable   │  │  M4A files   │
│  UI, reel    │  │ Symbols,     │  │ Free spins,  │  │              │
│  land, pay   │  │ big win,     │  │ picker,      │  │  Base game   │
│  line, spin  │  │ anticipate   │  │ hold & win   │  │  music only  │
│              │  │              │  │              │  │              │
│  ~500 KB     │  │  ~2 MB       │  │  ~2 MB       │  │  Per-file    │
│  Priority 1  │  │  Priority 2  │  │  Priority 3  │  │              │
└──────────────┘  └──────────────┘  └──────────────┘  └──────────────┘
```

Pool Card Features:

- Loading strategy badge (IMMEDIATE / DEFERRED / LAZY / MUSIC)
- SubLoader ID badge (A, B, C... for deferred pools)
- UNLOADABLE badge (for bonus pool)
- Sound count and estimated compressed size
- Size progress bar with percentage (red if over `maxSizeKB` limit)
- Expandable sound list with drag-and-drop between pools
- Settings: maxSizeKB, SubLoader selector (A–F, Z), unloadable checkbox

Auto-Assign Algorithm: Priority-ordered pattern matching, first match wins:

PATTERN	TARGET POOL	EXAMPLES
`BaseGameMusicLoop*`, `AmbBg`	standalone	BaseGameMusicLoop1, AmbBg
`Ui*`, `ReelLand*`, `SpinsLoop*`, `Payline*`	loading	UiSpin, ReelLand1, SpinsLoop1
`Bonus*`, `FreeSpin*`, `Picker*`, `HoldAnd*`	bonus	BonusIntro, FreeSpinMusic, PickerHit
`BigWin*`, `Sym*`, `Anticipation*`	main	BigWinStart, SymHp1Win, Anticipation
Everything else	main (fallback)	—

Copy Snippet Feature: For each deferred pool, generates integration code that game developers copy-paste:

```
// SubLoader A — Main pool (call on first spin)
soundManager.startSubLoader("A");

// SubLoader B — Bonus pool (call when bonus confirmed)
soundManager.startSubLoader("B");

// Unload B — After bonus ends
soundManager.unloadSubLoader("B");
```

4.5 Commands Page — Sound Command & Sprite List Editor

> *Visual editor for the `soundDefinitions` section of sounds.json. Manages commands (what the game calls), sprite lists (randomized/sequential groups), and validates against game source code.*

Layout: Two-tab view (Commands | Sprite Lists) → Filter bar → Scrollable list → Modals

Commands Tab:

A command is what the game calls via `soundManager.execute("commandName")`. Each command has one or more action steps.

Example command structure:

```
{
  "onBigWin": [
    { "command": "Stop",  "spriteId": "s_SpinsLoop1" },
    { "command": "Play",  "spriteId": "s_BigWinStart", "volume": 0.9 },
    { "command": "Play",  "spriteId": "s_CoinShower1", "volume": 0.7, "delay": 500 }
  ]
}
```

Command Row (collapsed):

- Command name (monospace)
- Action count badge
- Error badge (if references missing sprites)
- Hover actions: Copy, Rename, Delete

Command Row (expanded): Each action step shows:

FIELD	DISPLAY	EXAMPLE
Step #	Numbered badge	1, 2, 3
Command Type	Colored badge	Play, Stop, Fade, Set, Pause, Resume
Target	Sprite ID or List ID (color-coded)	s_BigWinStart, sl_ReelLands
Parameters	Inline badges	vol: 0.9, delay: 500ms, loop, overlap

Action Step Types:

TYPE	PARAMETERS	USE CASE
Play	volume, delay, loop, overlap, cancelDelay	Play a sound or random from list
Stop	—	Stop a playing sound
Fade	volume, duration, delay	Fade in/out (music transitions)
Set	volume, pan, rate	Modify a playing sound
Pause / Resume	—	Pause/resume a specific sound
Execute	—	Chain another command
ResetSpriteList	—	Reset a sprite list to beginning

Sprite Lists Tab:

A sprite list is a group of related sounds played randomly or sequentially:

```
{
  "sl_ReelLands": {
    "items": ["s_ReelLand1", "s_ReelLand2", "s_ReelLand3", "s_ReelLand4"],
    "type": "random",
    "overlap": true
  }
}
```

FIELD	OPTIONS
Type	`random` (pick one randomly) or `sequential` (cycle through)
Overlap	`true` = can play while previous is still playing
Items	Array of sprite IDs

Game Code Scanner: Scans the game repository's TypeScript source for `soundManager.execute()` calls and reports:

```
Scan Results:
  ✓ 28 hooks found in game source code
  ⚠ 3 missing — hooks called in game but not defined in sounds.json:
      onSymbolWin, onBonusIntroComplete, onWheelSpin
  ⚠ 2 empty — defined in sounds.json but have no action steps:
      onGameLoad, onBaseToBonus
  ⚠ 4 dead — defined in sounds.json but never called by game:
      onOldFeature, onDeprecatedWin, onTestHook, onUnused

  Auto-fix options:
  ☑ Add 3 missing commands (generates stub actions from sprite patterns)
  ☑ Remove 4 dead commands
  ☑ Fill 2 empty commands with suggested actions
```

Generate Missing Commands: Auto-creates commands for sprites not yet referenced by any command:

- Analyzes sprite name patterns (e.g., `SymHp1Win` → suggests `onSymbolHp1Win`)
- Generates default Play action with appropriate volume
- Preview modal lets user include/exclude each suggestion before saving

---

4.6 Build Page — Audio Build, Deploy & Game Launch

> *The operational center. Chains build → validate → deploy → game build → launch in a single click with live streaming output.*

Layout: Two-column top (Audio Pipeline | Game Panel) → Full-width output log bottom

Left Column — Audio Pipeline:

SECTION	CONTENT
Build Scripts	Buttons for each npm script (build, build-audio, etc.)
After-Build Action	Dropdown: None, Deploy Only, Deploy + Launch
Dist Sprites	Badges showing built M4A files and sizes
Other Scripts	Additional npm scripts (validate, clean, etc.)

Right Column — Game Panel:

SECTION	CONTENT
Game Repo	Path display, Node version badge
Branch Selector	Dropdown to switch game branches
Launch Scripts	Grid of `playa launch` variants with Launch buttons
Game Git	Branch, changed files, commit + push + PR workflow

One-Click Build Chain:

```
User clicks "build" with After-Build set to "Deploy + Launch"

1. npm run build                 → buildTiered.js (WAV → M4A sprites)
   ├── SHA256 hash check         → skip unchanged sounds
   ├── Parallel FFmpeg encode    → loading.m4a, main.m4a, bonus.m4a
   └── Size check                → ⚠ OVER LIMIT warning (non-blocking)

2. npm run build-validate        → validateBuild.js (5 QA checks)
   ├── M4A files exist + sizes
   ├── Command → sprite refs valid
   ├── SpriteList → sprite refs valid
   ├── SoundSprite → manifest refs valid
   └── Orphan sprite detection

3. npm run deploy                → copyAudio.js
   ├── Clean game sounds/ dir
   └── Copy M4A + sounds.json → game/assets/.../sounds/

4. yarn install (game repo)      → if dependencies changed

5. yarn build-dev (game repo)    → webpack rebuild

6. playa launch (game repo)      → dev server on port 8080

7. Wait for port 8080            → TCP polling, 120s timeout

8. Open browser                  → Chrome/Edge with isolated profile
```

Live Output Log:

- Streams stdout/stderr in real-time via `script-output` IPC event
- Auto-scrolls to bottom
- Filters noisy npm warnings
- Stop button aborts the entire chain
- Pulsing green dot indicates "running"

Game Git Integration:

Built-in workflow for committing audio changes to the game repo:

```
1. User clicks "Refresh" → loads game git status
2. Auto-generates branch name: feature/PA-treasure-of-troy-audio-update
3. Auto-generates commit message: "Update 3 audio sprites, sounds.json"
4. Auto-selects target branch: develop (or release/*)
5. User clicks "Commit, Push & PR"
   → git checkout -b [branch]
   → git add -A
   → git commit -m [message]
   → git push -u origin [branch]
   → gh pr create --base [target] --title [title] --body [body]
6. PR URL displayed as clickable link
```

GLR Local Launch (No VPN):

- Lists pre-recorded GLR sessions from `game/GLR/` directory

- Each GLR subdir contains recorded RGS responses (authenticate, initstate, play.N)
- `playa launch --glr [name]` serves game with recorded data
- Common GLRs: `GLR` (full), `bonus`, `bigWin`, `wins`
- Auto git-pulls game repo before launch

4.7 Git Page — Audio Repository Version Control

> *Simple, focused git interface for the audio repository. Commit and push with auto-generated messages.*

Layout: Header with status badges → Two-column (Changed Files | Recent Commits) → Commit input

Header Badges:

- Branch name (purple badge)
- Changes count (orange badge) — or "Clean" (green badge) if no changes

Changed Files Panel: Each file shows a colored status badge:

BADGE	COLOR	MEANING
NEW	Cyan	Untracked file
ADD	Green	Newly staged
MOD	Orange	Modified
DEL	Red	Deleted
REN	Purple	Renamed

Recent Commits Panel: Last 10 commits with:

- Git hash (first 7 chars, accent color)
- Commit message (truncated)

Auto-Generated Commit Messages: Analyzes changed files and generates contextual messages:

```
Changed: 3 new WAVs, sprite-config.json, dist/

Generated: "Add 3 sounds, update sprite config, rebuild sprites"
```

```
Changed: sounds.json, dist/sounds.json

Generated: "Update commands, rebuild JSON"
```

Key Actions:

- `Refresh` — Reloads git status
- `Commit & Push` — git add -A → commit → push (single click)
- Enter key in commit message triggers push

5. Data Model

5.1 Project Structure (Audio Repository)

```
audio-repo/
├── package.json          ← npm scripts (build, validate, deploy)
├── settings.json         ← paths, game repo link
├── sounds.json           ← SOURCE OF TRUTH for sound definitions
├── sprite-config.json    ← tier configuration, encoding settings
├── sourceSoundFiles/     ← raw WAV files (40-70 files typical)
│   ├── BigWinStart.wav
│   ├── ReelLand1.wav
│   ├── BaseGameMusicLoop.wav
│   └── .deleted/         ← soft-deleted WAVs (restorable)
├── scripts/              ← build scripts (from template)
│   ├── buildTiered.js
│   ├── buildTieredJSON.js
│   ├── copyAudio.js
│   └── validateBuild.js  (+ 8 more)
├── dist/                 ← build output
│   ├── sounds.json       ← generated
│   └── soundFiles/
│       ├── loading.m4a
│       ├── main.m4a
│       └── bonus.m4a
└── node_modules/
```

5.2 JSON Schemas

settings.json — Project paths and game repo link:

```
{
  "gameProjectPath": "../treasure-of-troy",
  "JSONtemplate": "./sounds.json",
  "JSONtarget": "./dist/sounds.json",
  "SourceSoundDirectory": "./sourceSoundFiles",
  "DestinationSoundDirectory": "./dist",
  "DestinationAudioSpriteDirectory": "./dist/soundFiles"
}
```

sprite-config.json — Tier definitions and encoding:

```
{
  "spriteGap": 0.05,
  "sprites": {
    "loading": {
      "maxSizeKB": 700, "priority": 1,
      "sounds": ["UiSpin", "ReelLand1", "Payline", "SpinsLoop1"]
    },
    "main": {
      "maxSizeKB": 3000, "priority": 2, "subLoaderId": "A",
      "sounds": ["BigWinStart", "SymHp1Win", "Anticipation"]
    },
    "bonus": {
      "maxSizeKB": 3000, "priority": 3, "subLoaderId": "B", "unloadable": true,
      "sounds": ["FreeSpinMusic", "PickerMusicLoop", "BonusTransition"]
    }
  },
  "standalone": { "sounds": ["BaseGameMusicLoop", "AmbBg"] },
  "encoding": {
    "sfx":   { "bitrate": 64,  "channels": 1, "samplerate": 44100 },
    "music": { "bitrate": 128, "channels": 2, "samplerate": 44100 }
  }
}
```

sounds.json — Sound definitions (source of truth):

```
{
  "soundManifest": [
    { "id": "loading_sprite", "src": ["soundFiles/loading.m4a"] },
    { "id": "main_sprite", "src": ["soundFiles/main.m4a"], "loadType": "A" },
    { "id": "bonus_sprite", "src": ["soundFiles/bonus.m4a"], "loadType": "B", "unloadable": true },
    { "id": "BaseGameMusicLoop", "src": ["soundFiles/BaseGameMusicLoop.m4a"] }
  ],
  "soundDefinitions": {
    "soundSprites": {
      "s_ReelLand1": {
        "soundId": "loading_sprite", "spriteId": "ReelLand1",
        "startTime": 0, "duration": 2666,
        "tags": ["SoundEffects"], "overlap": false
      }
    },
    "commands": {
      "onReelLand": [
        { "command": "Play", "spriteId": "s_ReelLand1", "volume": 1.0 }
      ],
      "onBigWin": [
        { "command": "Stop", "spriteId": "s_SpinsLoop1" },
        { "command": "Play", "spriteId": "s_BigWinStart", "volume": 0.9 },
        { "command": "Play", "spriteId": "s_CoinShower1", "volume": 0.7, "delay": 500 }
      ]
    },
    "spriteList": {
      "sl_ReelLands": {
        "items": ["s_ReelLand1", "s_ReelLand2", "s_ReelLand3"],
        "type": "random", "overlap": true
      }
    }
  }
}
```

---

6. Tiered Audio Loading — playa-core SubLoader System

6.1 The Problem

A typical slot game has 40–70 individual sounds totaling 5–15 MB of compressed audio. Loading all audio at game start causes:

- **Slow initial load** — player waits 5–10 seconds before first spin
- **Wasted bandwidth** — bonus audio (40–60% of total) may never be needed
- **Memory pressure** — all audio in RAM for the entire session

6.2 The Solution: Tiered Audio Pools

```
┌──────────────────────────────────────────────────────────────────┐
│                          GAME TIMELINE                           │
│                                                                  │
│  Game Start        First Spin         Bonus Confirmed     Bonus  │
│      │                 │                    │              End   │
│      ▼                 ▼                    ▼                │   │
│  ┌────────┐     ┌────────────┐     ┌──────────────┐        │   │
│  │LOADING │     │   MAIN     │     │    BONUS     │        │   │
│  │ ~500KB │     │  ~2 MB     │     │   ~2 MB      │        │   │
│  │        │     │            │     │              │        │   │
│  │ No     │     │loadType:"A"│     │loadType:"B"  │   ┌────┴──┐│
│  │loadType│     │ Deferred   │     │Deferred +    │   │UNLOAD ||
│  │        │     │            │     │Unloadable    │   │bonus  ||
│  │Loaded  │     │startSub-   │     │startSub-     │   │pool   ||
│  │with    │     │Loader("A") │     │Loader("B")   │   │free RAM||
│  │game    │     │on 1st spin │     │on 3+ scatter │   │       ||
│  └────────┘     └────────────┘     └──────────────┘   └───────┘│
│                                                                  │
│  ┌──────────────────────────────────────────────────────────┐   │
│  │ STANDALONE — Base game music, each = separate M4A file,    │   │
│  │ clean Howler.js looping (no micro-gaps)                    │   │
│  └──────────────────────────────────────────────────────────┘   │
└──────────────────────────────────────────────────────────────────┘
```

6.3 loadType Reference

LOADTYPE	PLAYA-CORE BEHAVIOR	USE CASE
(absent)	Loaded with initial game bundle	UI, reel sounds, loading pool
`"A"` – `"F"`	Deferred SubLoader — on demand	Main pool (A), bonus pool (B)
`"Z"`	Lazy — loaded on first play	Rare sounds that may never play

6.4 SubLoader Queue & Timing

playa-core loads SubLoaders **one at a time** in FIFO order:

```
Frame 1: startSubLoader("A")  → A begins loading
Frame 2: startSubLoader("B")  → B queued (A still loading)
...
Frame N: A completes           → B begins loading
Frame M: B completes           → queue empty
```

Critical rule for game developers:

```
WRONG: startSubLoader("B") on scatter symbol landing on a reel
       ↑ Scatter can land without triggering bonus (need 3+)

RIGHT: startSubLoader("B") in BonusTriggerCommand, AFTER reel
       evaluation confirms 3+ scatters
       ↑ Bonus is guaranteed, intro animation buys loading time
```

---

7. Build Pipeline

7.1 Audio Build Flow

```
Source WAVs          buildTiered.js          dist/soundFiles/
(44.1kHz PCM)  ──→   Groups by tier    ──→   loading.m4a
                     SHA256 hash cache        main.m4a
                     Parallel FFmpeg          bonus.m4a
                                              soundData_*.json
                                                    |
                     buildTieredJSON.js             |
                     Reads soundData_*  ←──────────┘
                     Reads sounds.json (source of truth)
                     Generates dist/sounds.json
```

Key behaviors:

- **Incremental builds:** SHA256 hash cache — only changed sounds are rebuilt
- **Failed tier safety:** If a tier build fails, its hashes are NOT saved (forces rebuild)
- **Size warnings:** `maxSizeKB` exceedance logs warning, does NOT fail the build
- **Validation:** 5 QA checks — M4A existence, command refs, spriteList refs, soundSprite refs, orphans

7.2 Deploy Target

```
game-repo/assets/default/default/default/sounds/
├── sounds.json
└── soundFiles/
    ├── loading.m4a
    ├── main.m4a
    └── bonus.m4a
```

---

8. IPC Channel Reference

8.1 Project Management

CHANNEL	DIRECTION	PURPOSE
`open-project`	renderer→main	Dialog picker, returns loaded project
`reload-project`	renderer→main	Re-read project from disk
`save-sounds-json`	renderer→main	Write sounds.json
`save-sprite-config`	renderer→main	Write sprite-config.json
`save-settings`	renderer→main	Write settings.json
`health-check`	renderer→main	Validate project structure
`init-from-template`	renderer→main	Bootstrap from bundled template

8.2 Sound Management

CHANNEL	DIRECTION	PURPOSE
`import-sounds`	renderer→main	Multi-select WAV dialog
`delete-sound`	renderer→main	Soft delete to .deleted/
`restore-sound`	renderer→main	Restore from .deleted/
`list-deleted-sounds`	renderer→main	List .deleted/ contents
`analyze-orphans`	renderer→main	Find WAVs not in sounds.json
`clean-orphans`	renderer→main	Delete orphan WAVs

8.3 Build & Deploy

CHANNEL	DIRECTION	PURPOSE
`run-script`	renderer→main	Execute npm script (validated name)
`run-deploy`	renderer→main	Execute deploy script
`clean-dist`	renderer→main	Remove M4A + JSON from dist/
`script-output`	main→renderer	**Event** — live stdout/stderr streaming

8.4 Game Integration

CHANNEL	DIRECTION	PURPOSE
`pick-game-repo`	renderer→main	Directory picker
`configure-game`	renderer→main	Link game repo
`get-game-scripts`	renderer→main	Read playa launch scripts
`run-game-script`	renderer→main	Spawn game dev server
`build-game`	renderer→main	Run yarn build-dev
`yarn-install-game`	renderer→main	Run yarn install
`kill-game`	renderer→main	Kill game process
`pull-game-json`	renderer→main	Copy sounds.json from game
`git-pull-game`	renderer→main	Git pull game repo
`scan-game-hooks`	renderer→main	Scan TS for soundManager calls
`wait-for-port`	renderer→main	Poll TCP port until available
`list-glr`	renderer→main	List GLR sessions
`launch-local-glr`	renderer→main	Launch with pre-recorded data

8.5 Git Operations

CHANNEL	DIRECTION	PURPOSE
`git-status`	renderer→main	Branch, status, last 10 commits
`git-commit-push`	renderer→main	git add -A → commit → push
`game-git-status`	renderer→main	Game repo git status
`game-git-create-branch-commit-push`	renderer→main	Branch, commit, push game repo
`game-git-create-pr`	renderer→main	Create PR via gh CLI

9. Security Model

9.1 Process Isolation

```
Renderer (untrusted)          Main (trusted)
────────────────────          ──────────────────
contextIsolation: true        All file/process ops
nodeIntegration: false        IPC handlers validate
No Node.js APIs               every input before use
No require/import
window.api.* only
```

9.2 Input Validation

ATTACK VECTOR	MITIGATION
Shell injection via script name	Regex: `/^[a-zA-Z0-9_-]+$/`
Path traversal via filename	`path.basename()` + `.wav` check + `startsWith(sourceDir + path.sep)`
Shell injection via git	`execFileSync` with array args (no shell)
URL injection	`open-url` validates `http://` or `https://` only
Process hanging	Timeout on every subprocess (120s–300s)
Buffer overflow	`maxBuffer: 5MB` on all exec calls

9.3 Subprocess Strategy

FUNCTION	USE CASE	WHY
`exec()`	npm/yarn commands	Needs shell for PATH resolution
`execFileSync`	git commands	Array args prevent injection
`spawn()`	Long-running processes	Streaming output, kill support

10. Cross-Platform Compatibility

CONCERN	SOLUTION
Window chrome	`hiddenInset` (macOS) / `titleBarOverlay` (Windows)
Path separators	`path.join()` everywhere, `/[/\\]/` regex for display
Shell commands	`exec()` for npm, `execFileSync` for git
Game browser	Chrome/Edge detection with fallback
Drag region	`-webkit-app-region: drag` (Chromium)
File URLs	`pathToFileURL()` for Windows compatibility

11. UI Design System

11.1 Visual Language

TOKEN	VALUE	USAGE
Background	`#0a0a14`	Primary dark background
Card BG	`rgba(22,22,42,0.65)`	Glass morphism cards
Accent	`#8b7cf8`	Primary purple accent
Cyan	`#38bdf8`	Build/active states
Green	`#4ade80`	Success/deploy
Orange	`#fb923c`	Warnings
Danger	`#f87171`	Errors/delete

11.2 Components

- `.card` — Backdrop blur container with hover glow
- `.badge` — Small colored labels (`text-xs`)
- `.btn-primary` — Purple gradient with shadow
- `.btn-ghost` — Transparent with border
- `.input-base` — Dark input with accent focus ring
- `.section-label` — Uppercase tracked section titles

11.3 Animations

- `.anim-fade-up` — Page transitions (0.25s)
- `.anim-fade-in` — Element appearance (0.2s)
- `.anim-pulse-dot` — Live indicator (1.5s loop)

---

12. Build & Distribution

12.1 Development

```
npm run dev          # Vite dev server (port 5173) + Electron
```

12.2 Production

```
npm run build-win    # → release/SlotAudioManager-Setup.exe (NSIS)
npm run build-mac    # → release/SlotAudioManager.dmg
npm run build-all    # Both platforms
```

Bundled: `main.js`, `preload.js`, `dist-renderer/**`, `template/**`

Dependencies: ALL in `devDependencies` — electron-builder excludes devDeps from asar, keeping production bundle lean.

---

13. Key Design Decisions

DECISION	RATIONALE
No TypeScript	Rapid iteration, single developer, minimal build overhead
Always-mounted pages	Preserves state across navigation (build logs, forms, playback)
Root sounds.json as source of truth	User edits are authoritative; build reads, never overwrites
Auto-deploy after build	Eliminates manual step, reduces human error
Isolated browser profile	Clean game state every launch — no stale cache
`structuredClone` over spread	Prevents stale nested reference bugs in React state
Custom WAV decoder	`decodeAudioData` crashes Electron for 24/32-bit PCM
Soft delete (trash)	Safety net — accidental deletes are recoverable
Git integration in-app	Audio engineers stay in one tool, no terminal context-switching